CSS Layout Basics

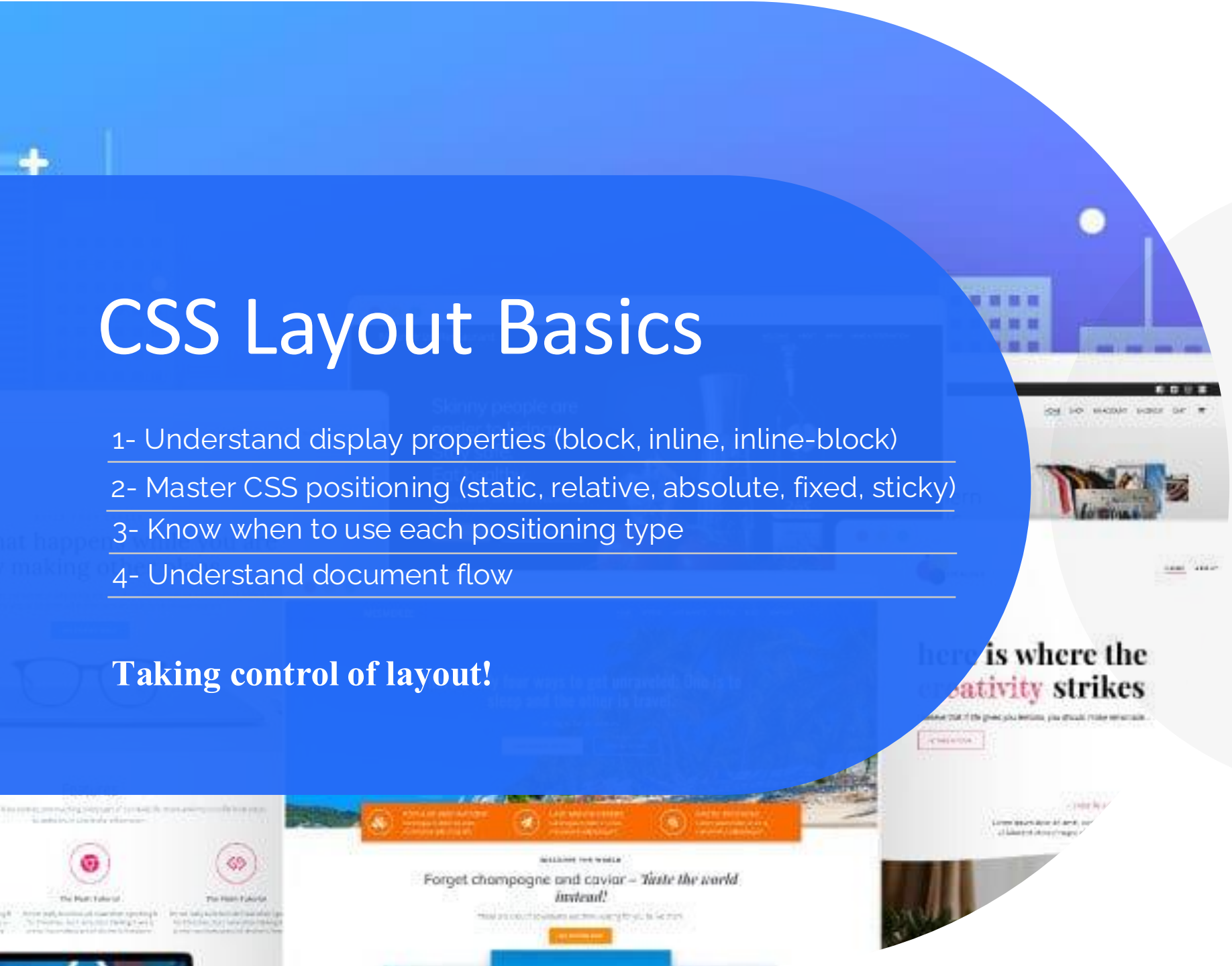
1- Understand display properties (block, inline, inline-block)

2- Master CSS positioning (static, relative, absolute, fixed, sticky)

3- Know when to use each positioning type

4- Understand document flow

Taking control of layout!



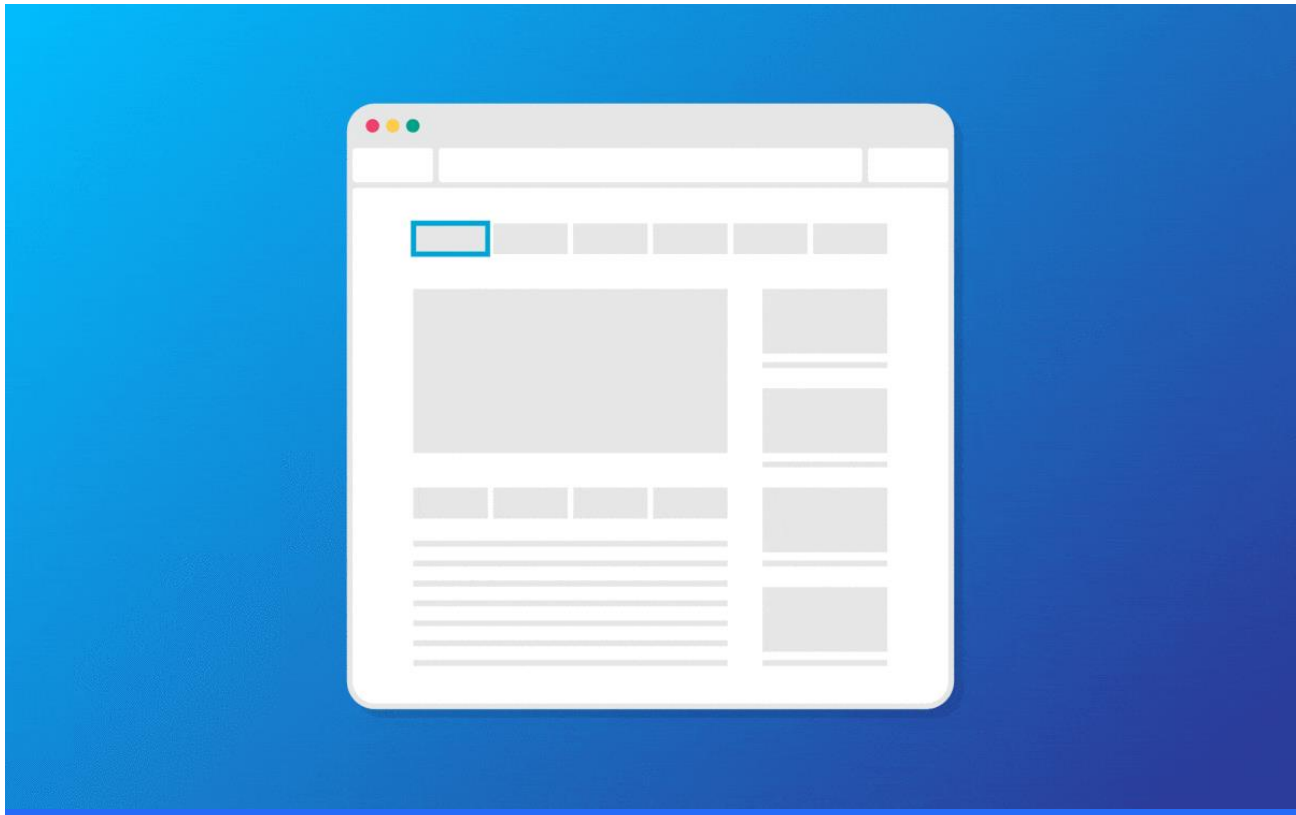
SCROLL

The Layout Problem

```
<header>Header</header>
<nav>Navigation</nav>
<main>Main Content</main>
<aside>Sidebar</aside>
<footer>Footer</footer>
```

By default, HTML stacks vertically...

Header
Navigation
Main Content
Sidebar
Footer



But what if we want them side-by-side?

What We Want?

Modern Web Layouts Need:



Navigation & Headers:

- Navigation bars (items side-by-side)
- Fixed headers that scroll with you (like Instagram/TikTok)
- Sticky search bars



Content Layouts:

- Sidebars next to content (news sites, blogs)
- Image grids (Pinterest, Instagram feeds)
- Card layouts (Netflix, YouTube)



Interactive Elements:

- Overlays and modals (pop-ups, lightboxes)
- Tooltips and dropdown menus
- Precisely positioned badges/notifications

Display: Block

Common Block Elements:

- **Content Structure:**
  `<div>`, `<p>`, `<h1>`-`<h6>`
- **Semantic Sections:**
  `<header>`, `<footer>`, `<article>`, `<section>`, `<nav>`, `<main>`
- **Lists:**
  `<ul>`, `<ol>`, `<li>`

Think of blocks as boxes stacked on top of each other.

```
div, p, h1, header, section {
  display: block;
}
```

Characteristics:

- Takes full width available
- Forces line break before and after
- Respects width, height, margins, padding
- Stacks vertically

Most structural elements are block by default!

Display: inline

Common Inline Elements:

- **Text Styling:**
  `<span>`, `<strong>`, `<em>`, `<mark>`
- **Interactive:**
  `<a>` (links), `<button>`*
- **Special Cases:**
  `<img>`* - *inline-block by default!*

They flow like text!
No line breaks!

```
span, a, strong, em {
  display: inline;
}
```

Characteristics:

- Only takes space it needs
- NO line breaks (flows horizontally)
- IGNORES width and height
- IGNORES top/bottom margins
- Flows with text (like words in a sentence)

Opposite of block!

Display: inline-block

Perfect for:

- Navigation items
- Buttons
- Image galleries
- Cards in a row

This is the hybrid - it flows but gives you control! It's like inline (flows horizontally) BUT you can set width, height, and all spacing like block

```
.element {
  display: inline-block;
}
```

Characteristics:

- Flows inline (side-by-side)
- BUT respects width, height, all margins/padding
- Elements can sit next to each other
- You control their size

Inline-Block: Best of Both!

Display: none

```css
/* Hidden - Not Displayed: */
.hidden {
  display: none;
}

/* The Differences */
article.hidden-example {
    display: none;       /* Gone completely, no space */
    visibility: hidden; /* Invisible, BUT space remains */
}
```

Common Uses:

- Mobile navigation menus (hidden until clicked)
- Modals/pop-ups (hidden until triggered)
- Tab content (show one, hide others)
- You'll use this with JavaScript soon!

Display none makes things vanish completely - no space left behind.

What It Does:

- Completely removes element from page
- Takes up NO space (like it doesn't exist)
- Other elements move up to fill the gap
- Perfect for show/hide functionality

You'll use this ALL THE TIME with JavaScript for interactive elements - hidden menus, pop-ups, tabs. For now, just know it exists!"

Display Demo

Block (Blue)

- Stacks vertically (each on own line)
- Takes full width (stretches across)
- Width is set, but doesn't matter much

Block 1

Block 2

Block 3

Inline (Red)

- Flows horizontally (all on same line!)
- Only takes space it needs
- Width setting is IGNORED

Inline 1 Inline 2 Inline 3

Inline-Block (Green)

- Flows horizontally (side-by-side!)
- Respects the width (150px boxes)
- Magic combo!

Box 1 Box 2 Box 3

Common Use Cases

Block - Structural elements

- Sections, containers, divs
- Headers, footers, articles
- Paragraphs, headings
- Anything that needs full width

Inline - Text-level elements

- Links within paragraphs
- Bold, italic, emphasis
- Spans for styling parts of text
- Small icons next to text

Inline-Block - Hybrid layouts

- Navigation menu items
- Button groups
- Image galleries
- Cards/boxes in a row
- Before Flexbox, this was THE solution!

None - Hide completely

- Hidden content (for JavaScript later)
- Mobile vs desktop toggles

CSS POSITIONING



Position: static

```css
.element {
  position: static; /* default */
}
```

What It Means:

- Element in normal document flow
- Stacks like usual (block stacks, inline flows)
- top, right, bottom, left have NO effect
- You rarely set this explicitly

Every element you've coded so far = static positioning

This is what EVERYTHING starts as!

Position: relative

```css
.element {
  position: relative;
  top: 20px;    /* Move DOWN 20px from normal position */
  left: 30px;   /* Move RIGHT 30px from normal position */
}
```

Key Points:

- Element STAYS in document flow (space preserved!)
- Moves relative to its NORMAL position
- Leaves a "ghost" - space remains where it was
- Other elements don't see the offset
- Important: Creates reference point for absolute children

The element moves visually, but it leaves a 'ghost' space behind where it originally was. Other elements act like it's still there.

Position: relative - Examples

HTML:

```html
<div class="box">Normal Box 1</div>
<div class="box relative-box">Relative Box (moves!)</div>
<div class="box">Normal Box 3</div>
```

CSS:

```css
.relative-box {
  position: relative;
  top: 20px;  /* Moves DOWN 20px */
  left: 30px; /* Moves RIGHT 30px */
  background-color: red;
}
```

What You'll See:

- Red box moves down and right
- **GHOST SPACE** remains where it was!
- Box 3 stays below the ghost (not the red box!)
- **Notice the space left behind!**

Position: absolute

```css
.element {
  position: absolute;
  top: 50px;
  right: 20px;
}
```

Key Points:

- REMOVED from document flow (no space left!)
- Positioned relative to nearest POSITIONED ancestor
- If no positioned ancestor → uses <body>
- Use top/bottom/left/right to place precisely
- NO ghost space - other elements move up

CRITICAL RULE:
"Absolute" is positioned relative to its closest parent with `position: relative` *(or absolute/fixed)*

Most powerful, most confusing - watch closely!

Position Absolute - The Trick

```css
.parent {
  position: relative; /* This is the reference! */
  width: 300px;
  height: 200px;
  background-color: lightblue;
}

.child {
  position: absolute;
  top: 20px;
  right: 20px;
  background-color: orange;
  width: 100px;
  height: 50px;
}
```

This is THE most important positioning concept!

How It Works:

1. Child looks for nearest **positioned** parent
2. Parent must have `position: relative/absolute/fixed` (NOT static!)
3. Child positions itself relative to **THAT parent**
4. No positioned parent? → Uses `<body>` (entire page!)

The Pattern:

Parent: position: relative (becomes the reference frame)
└— Child: position: absolute (positions inside parent!)

Absolute Common Use Cases

Badges & Notifications

- Notification count on app icons (like Instagram's red circle)
- "New" badges on products
- Sale tags on corners

Overlays & Pop-ups

- Modal dialog boxes
- Tooltips that appear on hover
- Dropdown menus
- Cookie consent banners

Precise Placement

- Close (X) buttons on modals
- Icons overlaid on images
- Watermarks on photos
- Profile badges/verified checkmarks

Design Effects

- Decorative elements
- Background patterns
- Floating action buttons

Position: fixed

```css
.element {
  position: fixed;
  top: 0;
  left: 0;
}
```

Fixed Positioning - Stays Put When You Scroll!

Key Points:

- REMOVED from document flow (like absolute)
- Positioned relative to VIEWPORT (browser window)
- STAYS IN PLACE when you scroll (This is the magic!)
- Perfect for headers, navbars, chat buttons

Common Examples You See Daily:

- Sticky headers on websites (scroll down, header stays!)
- Fixed navigation bars (Instagram, TikTok)
- Chat/help buttons (floating bottom-right corner)
- "Back to Top" buttons
- Cookie consent bars

Viewport = Your Browser Window

Don't forget: Add padding to <body> so content doesn't hide under fixed header!

Position Fixed - Example

```css
/* IMPORTANT: Prevent content from hiding! */
body {
    padding-top: 60px; /* Height of nav bar */
}

.fixed-nav {
    position: fixed;
    top: 0; /* Stick to top of viewport */
    left: 0; /* Start at left edge */
    right: 0; /* Stretch to right edge */
    background-color: #2c3e50;
    padding: 20px;
    z-index: 1000; /* Keep on top of everything! */
}
```

The z-index: 1000 makes sure it stays ON TOP of everything else - we'll cover z-index more soon

What This Does:

1. Nav bar fixes to top of viewport
2. Stays there when you scroll
3. z-index: 1000 keeps it on top of other elements
4. Body padding prevents content from hiding under n

Position: sticky

Key Points:

- Starts as relative - flows normally in document
- Scrolls with the page like normal... until...
- Hits the threshold (top: 0) - THEN it sticks!
- Acts like fixed while scrolling
- Unsticks when parent container scrolls past

```
.element {
  position: sticky;
  top: 0; /* Threshold - where it "sticks" */
}
```

This is newer and SUPER useful. Watch this magic! Sticky starts as position: relative - normal flow.

Best of Both Worlds:

- Stays in document flow (unlike fixed)
- Sticks when scrolling (unlike relative)
- No body padding needed!

Perfect For:

- Section headers that stick while scrolling their section
- Table headers (scroll through data, header stays visible)
- Sidebar navigation
- Shopping cart "Add to Cart" buttons

Z-Index Explained

Key Points:

- Only works on POSITIONED elements (relative, absolute, fixed, sticky - NOT static!)
- Higher number = closer to you (on top)
- Lower number = further away (behind)
- Default is `auto` (usually 0)
- Negative values push elements behind (z-index: -1)

```
.element {
  position: absolute; /* Must be positioned! */
  z-index: 10;        /* Higher = on top */
}
```

Think: Layers in Photoshop - higher numbers stack on top!

Best of Both Worlds:

- 1-9 → Slightly above normal content
- 10-99 → Tooltips, dropdowns
- 100-999 → Modals, overlays
- 1000+ → Fixed headers, critical UI

Perfect For:

- Fixed nav: z-index: 1000 (always on top!)
- Modal overlay: z-index: 500 (covers page content)
- Tooltips: z-index: 100 (above normal elements)
- Background decorations: z-index: -1 (behind everything)

Z-Index Explained

What's In Document Flow?
Document Flow = Normal stacking/spacing of elements

Position	In Flow?	Space Reserved?	Notes
static	Yes	Yes	Default - normal behavior
relative	Yes	Yes	Moves visually, space stays
absolute	No	No	Removed - other elements move up
fixed	No	No	Removed - scrolls with viewport
sticky	Yes*	Yes*	In flow until it sticks

What "In Flow" Means:

- **In Flow** = Other elements "see" it and make space for it
- **Out of Flow** = Other elements act like it doesn't exist

Key Takeaway:

- **Relative** = Moves but leaves ghost space (still in flow)
- **Absolute/Fixed** = Disappears from flow completely (no space left)
- **Sticky** = In flow until threshold, then acts fixed

FLOAT & CLEAR



The Old Way - Float and Clear

Float & Clear - The Old-School Layout Method
Before Flexbox and Grid, we used float!

Historical Context:

- Used from ~2000-2015 for page layouts
- Frustrating, hacky, caused many headaches
- **2015:** Flexbox arrives - everything changes!
- **Today:** Rarely used for layout (Flexbox/Grid instead)

Why Learn It Now?

- You'll see it in **older code** (lots of legacy sites)
- Understand **CSS evolution** (appreciate modern tools!)
- Still useful for **text wrapping images** (original purpose!)
- Job interview questions sometimes mention it

Good news: You'll barely use this!
But you need to recognize it!

```css
.element {
    float: left; /* or right */
    width: 200px;  /* specify width */
    height: 100px;  /* specify height */
    margin: 10px;  /* space around the element */
    padding: 10px; /* space inside the element */
    border: 1px solid #000; /* optional border */
}
```

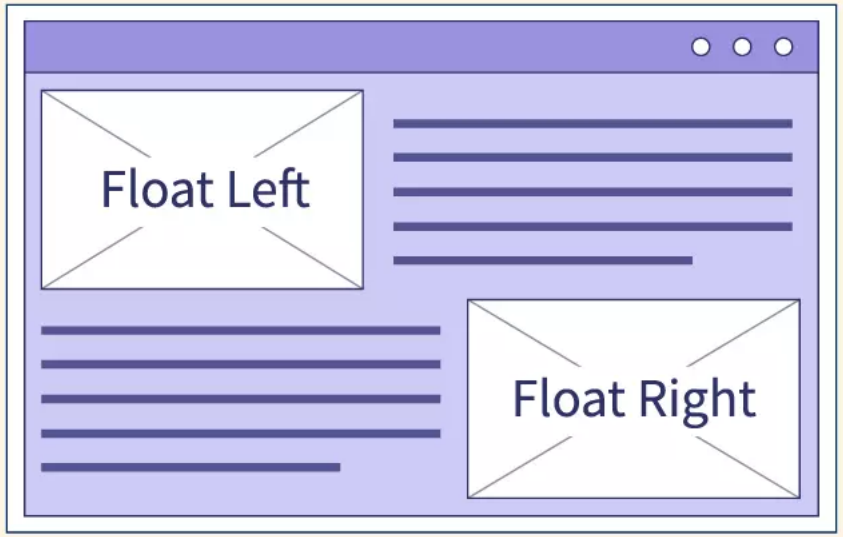
Float Basics

Originally designed for magazine-style text wrapping!

What Float Does:

- Removes element from normal flow (partially)
- Pushes it to left or right side of container
- Other content WRAPS around it
- Element becomes as wide as its content (unless width set)

```css
.element {
    float: left; /* or right */
    width: 200px;  /* specify width */
    height: 100px;  /* specify height */
    margin: 10px;  /* space around the element */
    padding: 10px; /* space inside the element */
    border: 1px solid #000; /* optional border */
}
```



Think: Magazine articles with images and text flowing around them!

Float Example

HTML:

```html
<article>
    <img src="dog.png" class="float-left" alt="Demo">
    <h1>This is a dog!</h1>
    <p>This text wraps around the floated image.
        Lorem ipsum dolor sit amet, consectetur adipiscing elit.
        The text flows naturally around the image, just like
        in magazines and newspapers!</p>
    <p>More text here...</p>
</article>
```

CSS:

```css
.float-left {
  float: left;
  margin: 0 20px 20px 0;  /* Space on right & bottom! */
  width: 300px;
}
```

The Margins Explained:

- `margin-right: 20px` → Space between image and text
- `margin-bottom: 20px` → Space below image
- `margin-top/left: 0` → Flush to edges

Perfect for:

- Blog posts with images
- News articles
- Magazine-style content
- Anywhere text should wrap around images!

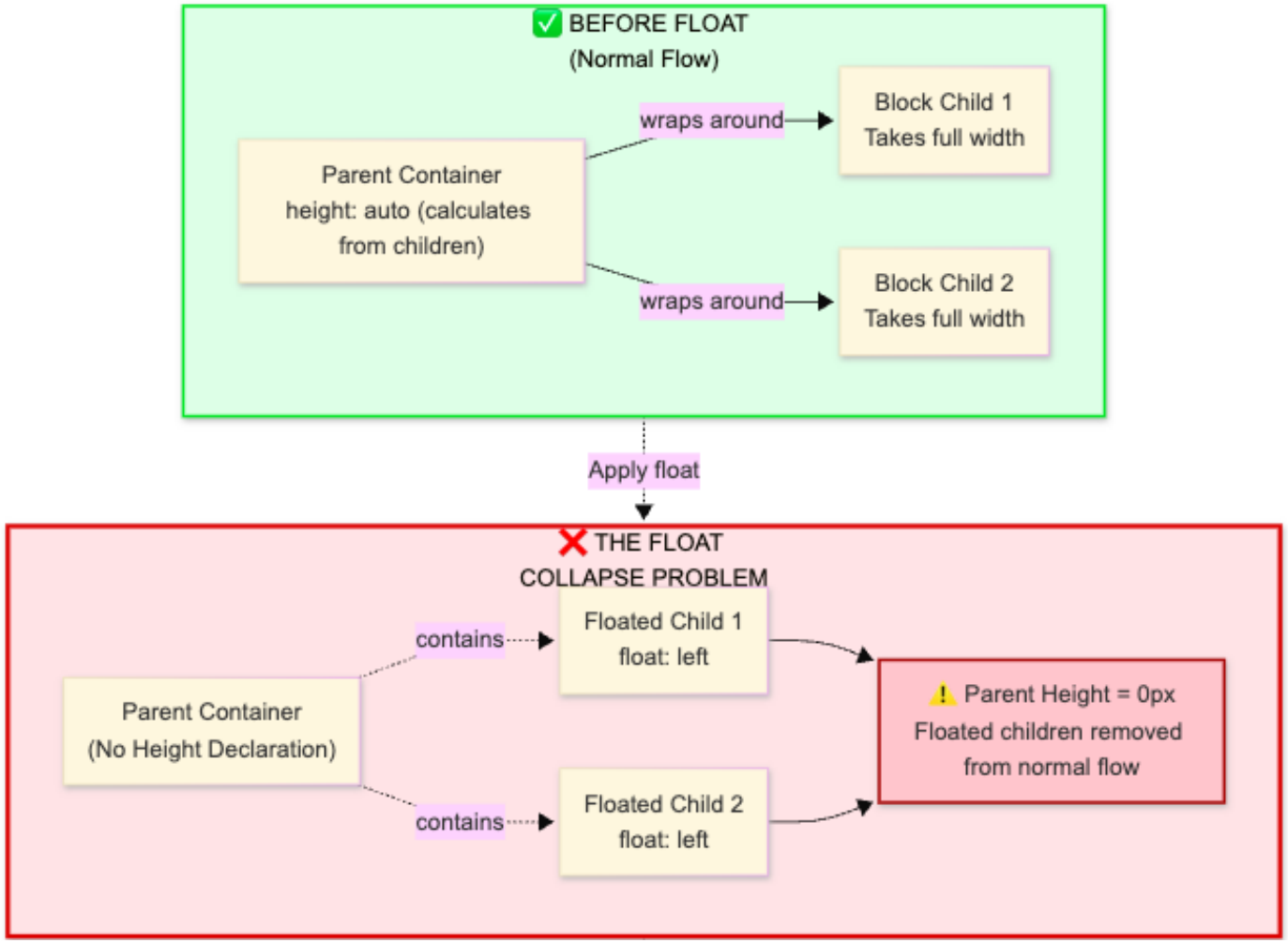
The Float Problem

The Problem:

- **Parent containers COLLAPSE** (don't contain floated children!)
- Elements "escape" their containers
- Height becomes 0 - background/borders disappear
- Layouts break in weird, unpredictable ways
- Requires hacky fixes (clearfix, overflow tricks)

This frustrated developers for YEARS!

Originally designed for magazine-style text wrapping!



Clear Property

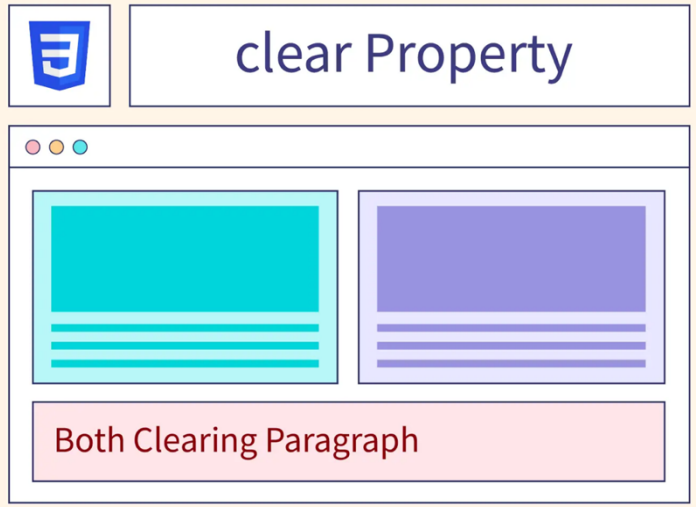
```css
.clear {
  clear: both;  /* or left, or right */
}
```

What Clear Does:

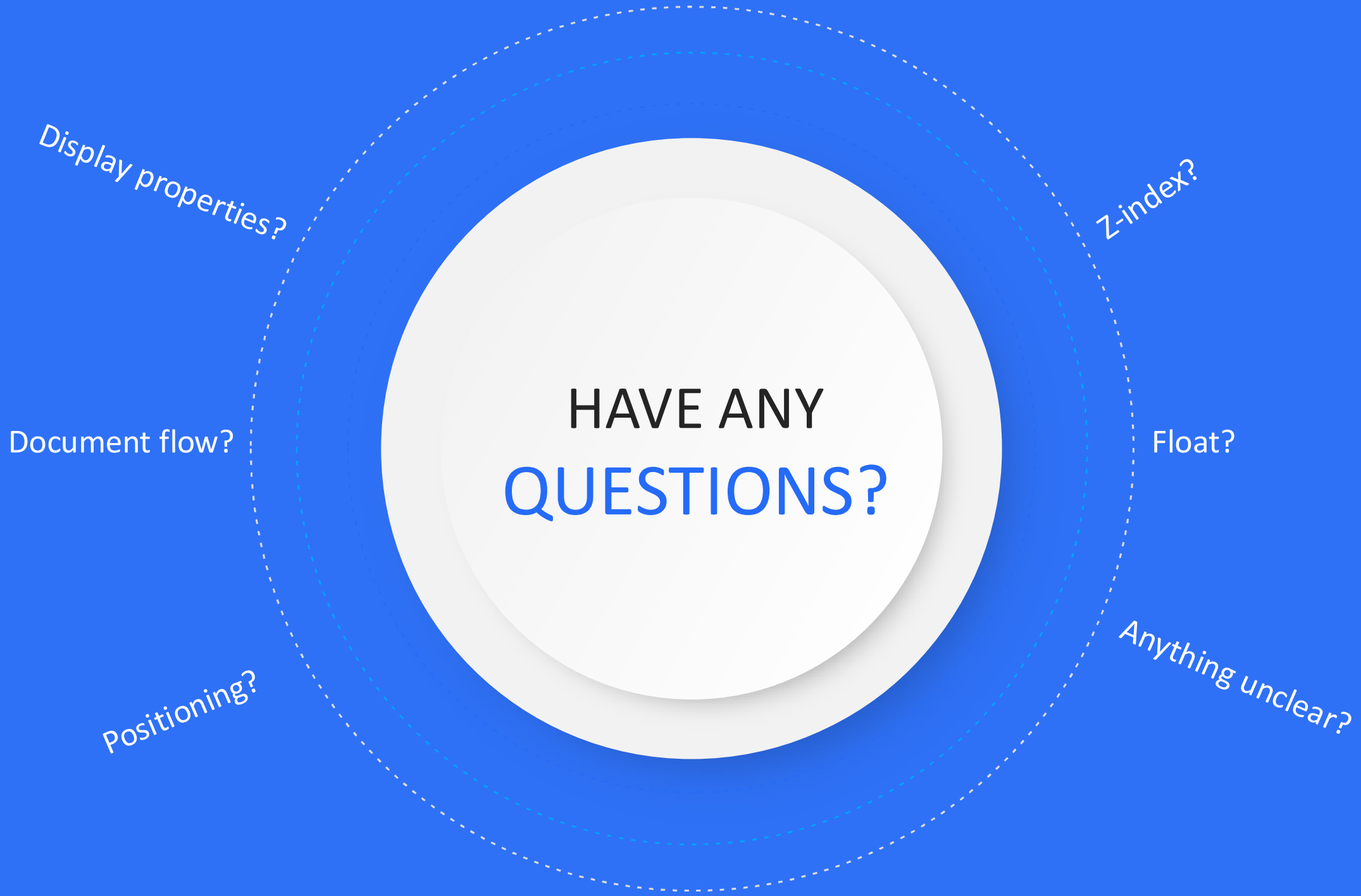
- Moves element **below** floated elements
- Prevents wrapping around floats
- "Clears" the float on specified side(s)
- Forces element to start below ALL floats

Values:

```css
clear: left;   /* Clear left-floated elements */
clear: right;  /* Clear right-floated elements */
clear: both;   /* Clear ALL floats (most common) */
clear: none;   /* Default - allow wrapping */
```



Helps with some float issues, but doesn't fix collapsing parent!


Display properties?
Z-index?
Document flow?
Float?
Positioning?
Anything unclear?
HAVE ANY
QUESTIONS?